IA3 DATA SCIENCE PROJECT REPORT

Predictive Maintenance Intelligence

Machine-failure prediction from sensor data, with a multi-machine factory simulation and monitoring dashboard

Student name: [YOUR NAME]

Register number: [REGISTER NUMBER]

Course: [COURSE NAME & CODE] Department: [DEPARTMENT]

Faculty: [FACULTY NAME] Date: October 2026

Abstract

Unplanned machine failure is expensive, yet failures are rare, so they are hard to learn. This project builds a complete predictive-maintenance pipeline on the UCI AI4I 2020 dataset (10,000 records, 3.39% failures). Six sensor readings are used to estimate each machine's failure probability. A leak-free protocol is followed: failure-cause label columns are excluded, model selection uses cross-validation on a training split, the alert threshold is chosen on a separate validation split, and a held-out test split is used once.

Three physics-based features (temperature gap, power, overstrain) lift the cross-validated PR-AUC of Gradient Boosting from 0.80 to 0.90. On the held-out test set the tuned model reaches precision 96.6%, recall 83.8%, F1 89.8% and PR-AUC 0.901, against F1 75.2% and PR-AUC 0.782 for the untuned baseline. Error analysis shows that failures with a physical signature (power, overstrain, heat dissipation) are detected almost perfectly, whereas random failures and the random part of tool-wear failures are not predictable from the sensors. The model is deployed in a Streamlit dashboard driven by a five-machine simulator, with scenarios, risk trajectories, adjustable alert sensitivity and rule-based explanations. The system is covered by 35 automated tests.

Keywords: predictive maintenance, imbalanced classification, gradient boosting, feature engineering, PR-AUC, threshold selection, Streamlit.

1. Introduction and Problem Statement

Industrial machines such as lathes and milling machines degrade through tool wear, overheating and overload. If a failure happens unexpectedly the production line stops; if maintenance is performed too often, money and machine time are wasted. Predictive maintenance aims to service a machine only when the data indicate that it is at risk.

1.1 Problem statement

Given the machine type and five sensor readings (air temperature, process temperature, rotational speed, torque and tool wear), predict the probability that the machine will fail, and raise an alert when the probability crosses a justified threshold.

1.2 Why the problem is hard

- **Severe class imbalance.** Only 339 of 10,000 records are failures (3.39%). A model that always predicts “no failure” scores 96.6% accuracy but detects nothing, so accuracy is not used as the main metric.
- **Unequal error costs.** A missed failure (false negative) is usually far more costly than an unnecessary inspection (false positive), so the decision threshold matters as much as the model.
- **Leakage risk.** The dataset contains five columns that describe the cause of each failure. They must not be used as inputs, or the task becomes trivial and the results meaningless.

1.3 Objectives

- Build and rigorously evaluate a failure-prediction model without data leakage.
- Improve on a baseline model through physics-based feature engineering and tuning.
- Choose and justify the alert threshold using validation data only.
- Analyse where the model succeeds and fails, with confidence intervals.
- Deliver a working multi-machine simulation and dashboard that demonstrates the model.

2. Dataset Description and Evaluation

The AI4I 2020 Predictive Maintenance Dataset (Matzka, 2020) is hosted in the UCI Machine Learning Repository (id 601). It contains 10,000 synthetic records that were generated to reflect the behaviour of real milling machines. The data have no missing values and no duplicate rows (verified in validate_dataset.py).

Table 1: Dataset columns and how each is used

Column	Role	Description
Type	Input	Product quality variant: L (60%), M (30%), H (10%)
Air temperature [K]	Input	Ambient temperature, 295.3–304.5 K
Process temperature [K]	Input	Process temperature, 305.7–313.8 K
Rotational speed [rpm]	Input	1,168–2,886 rpm, mean about 1,539

Column	Role	Description
Torque [Nm]	Input	3.8–76.6 Nm, mean about 40
Tool wear [min]	Input	0–253 min of accumulated tool use
Machine failure	Target	1 if the machine failed (339 records, 3.39%)
TWF, HDF, PWF, OSF, RNF	Excluded	Failure-cause indicators; excluded to prevent target leakage

2.1 Suitability of the dataset

The dataset suits the problem for four reasons: it is public and citable; its inputs are exactly the quantities a real machine would report; its failure rate and imbalance are realistic; and its documentation explains how failures arise, which allows physics-based feature engineering and a meaningful error analysis by failure mode.

2.2 Limitations of the dataset

- The data are synthetic, so the achieved scores are probably optimistic compared with real equipment.
- Two failure modes are random by construction (RNF, and TWF at a random wear level between 200 and 240 min), which caps the achievable recall.
- Only 339 failures exist, so the test set contains just 68, and the metrics carry substantial uncertainty (quantified in Section 6.3).

2.3 Data split

A stratified split keeps the failure rate near 3.4% in every part. The test set is set aside first and is not used for any decision.

Table 2: Stratified 60/20/20 data split

Split	Records	Failures	Failure rate
Train	6,000	203	3.38%
Validation	2,000	68	3.4%
Test	2,000	68	3.4%

3. Exploratory Data Analysis

The exploratory analysis (eda.py) confirms the imbalance and shows which readings separate failures from normal operation.

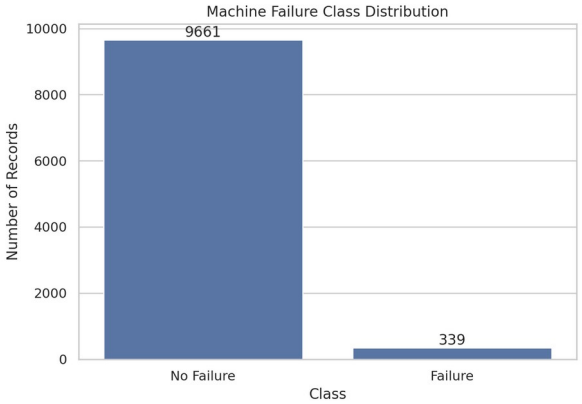


Figure 1: Class distribution: failures are only 3.39% of the records.

Sensors. Failed machines have a higher average torque (50.2 Nm versus 39.6 Nm) and more tool wear (143.8 min versus 106.7 min), and run slightly slower (1,496 rpm versus 1,540 rpm). Temperatures differ little between the two groups, which is why a single raw temperature is a weak predictor.

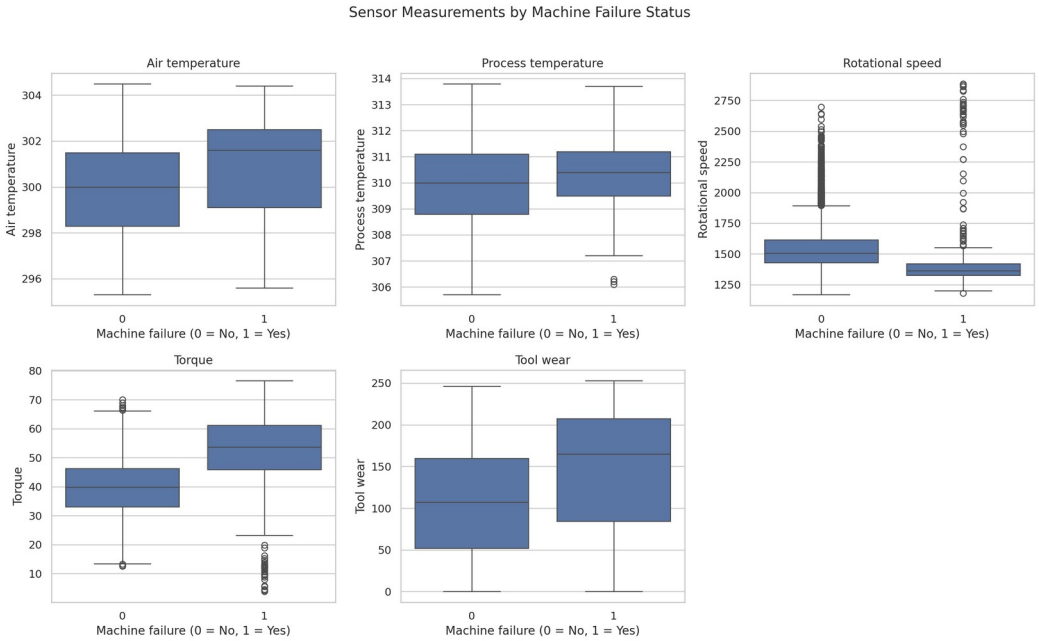


Figure 2: Sensor readings by failure status.

Failure modes. The dataset labels five causes: heat dissipation (HDF, 115), overstrain (OSF, 98), power (PWF, 95), tool wear (TWF, 46) and random (RNF, 19). Most failures therefore have a physical explanation based on combinations of sensors, not on any single sensor.

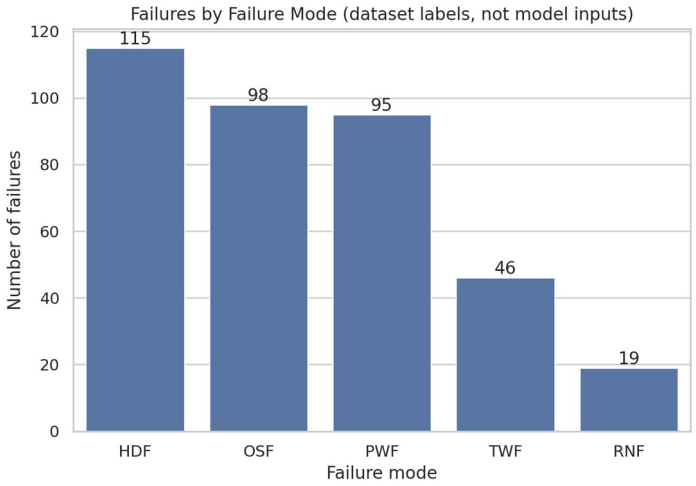


Figure 3: Failures by documented failure mode (labels, not model inputs).

Machine type. Failure rates are 3.92% for type L, 2.77% for M and 2.09% for H, so quality type carries a small amount of signal.

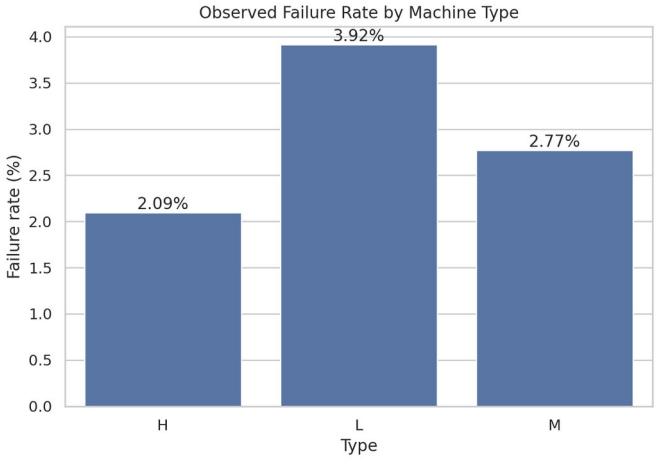


Figure 4: Observed failure rate by machine type.

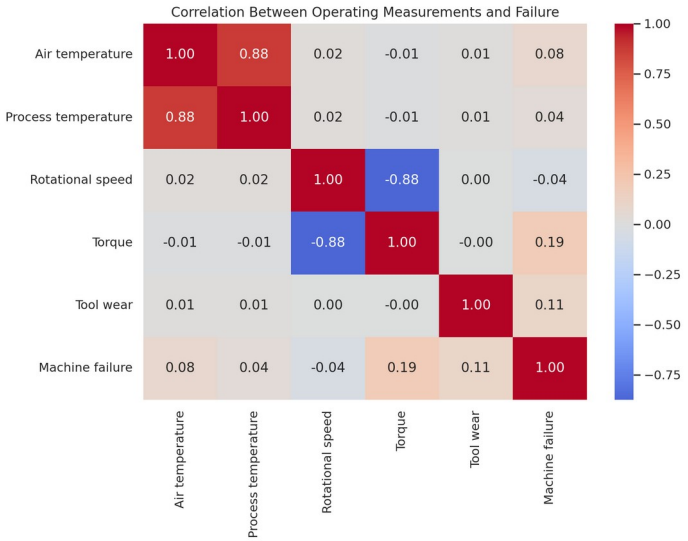


Figure 5: Correlation of the raw sensors with failure.

The correlation heat-map shows only moderate linear relationships with failure (torque the strongest), and a strong negative correlation between torque and rotational speed. Together with the failure-mode analysis this motivates non-linear models and engineered features.

4. Methodology

4.1 Physics-based feature engineering

The dataset documentation describes the conditions under which each failure mode occurs. Rather than asking the model to rediscover these combinations from raw columns, three features were computed from the six allowed inputs only (features.py):

- **Temperature difference** = process temperature − air temperature. Heat-dissipation failures occur when this gap is small while the machine runs slowly.
- **Power** = torque × rotational speed × 2π/60 (watts). Power failures occur when power is too low or too high.
- **Overstrain** = tool wear × torque. Overstrain failures occur when this product exceeds a limit that depends on the machine type.

These features use no failure labels, so they do not leak the target. They are part of the saved scikit-learn pipeline, so the dashboard applies exactly the same transformation at prediction time.

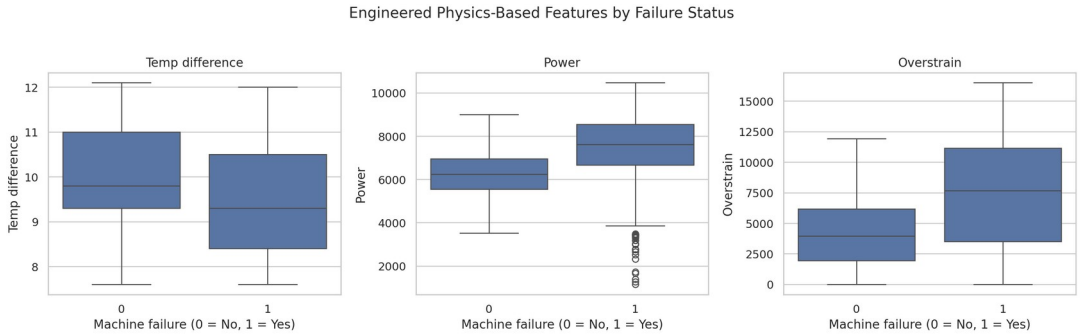


Figure 6: Engineered features by failure status: failed machines draw more power and show far higher overstrain.

4.2 Leak-free evaluation protocol

Every decision is made on data that the final test does not touch:

Table 3: Role of each data split

Split	Share	Used for
Training	60%	Fitting the models; 5-fold cross-validation for model comparison and hyper-parameter tuning
Validation	20%	Choosing the decision thresholds only
Test	20%	Final reporting, used once; also bootstrap confidence intervals and error analysis

4.3 Models, tuning and threshold selection

A dummy baseline, Logistic Regression, Random Forest and Gradient Boosting were each trained with and without the engineered features, all inside a pipeline with median imputation, standardisation and one-hot encoding. Class imbalance was handled with balanced class weights for Logistic Regression and Random Forest; Gradient Boosting is not class-weighted and instead relies on a tuned decision threshold. Models were ranked by cross-validated PR-AUC (average precision), which is more informative than ROC-AUC on imbalanced data (Saito and Rehmsmeier, 2015).

The best model family, Gradient Boosting with engineered features, was tuned by randomised search (16 candidates, 5-fold cross-validation, PR-AUC). The selected hyper-parameters were 100 trees, learning rate 0.05, depth 3, subsample 0.85 and minimum leaf size 5.

Two operating thresholds were then selected on the validation set. The balanced threshold (0.32) maximises F1. The high-recall threshold (0.05) is the highest threshold that still detects at least 85% of validation failures. Minimising an assumed 10:1 cost directly was tried, but it selected an unstable threshold of 0.03 on this small validation set and was rejected.

5. Implementation

The project is implemented in Python 3.12 with pandas, scikit-learn, matplotlib/seaborn and Streamlit. The code is organised so that the model, the simulation and the dashboard are separate, individually testable modules.

Table 4: Main modules

Module	Responsibility
features.py	Physics-based feature engineering shared by training and inference
rigorous_evaluation.py	Cross-validation, tuning, thresholds, final test, bootstrap, charts, model export
inference.py	Cached model loading; single and batch prediction; three-level risk status
explain.py	Failure-mode warnings that tell the operator why a machine is flagged
simulation/ machine_engine.py	Single-machine simulator with stress controls, tool replacement and reset
simulation/factory_*.py	Five-machine controller and monitor that links sensors to predictions
dashboard_logic.py, app.py	Streamlit dashboard with four pages
tests/	35 automated tests for the simulator, model, explanations and dashboard logic

5.1 Factory simulation

The simulator generates live readings for five machines of types L, M and H. Three controls (temperature stress, load stress and tool-wear rate) move each reading towards an operating target with smoothing and noise. Readings are clamped to the ranges seen in training, so the model is never asked to extrapolate to physically impossible values. A maintenance action

replaces the tool and resets wear to zero. Five one-click scenarios (normal operation, worn tool under load, heavy overload, overheating, light duty) make behaviour easy to demonstrate.

5.2 Dashboard

- **Factory Overview:** risk cards for all machines, advance by 1, 10 or 50 steps, risk-over-time chart and readings table.
- **Simulation Controls:** scenarios, sliders, tool replacement, per-machine risk trajectory and plain-language reasons for any alert.
- **Analytics:** dataset overview, model comparison, final metrics with confidence intervals, diagnostic charts and downloadable simulation history.
- **About & Limitations:** problem, method and honest limitations.

The sidebar lets the user switch between the balanced and high-recall thresholds. Machines are shown as LOW RISK, WATCH (probability above half the threshold) or HIGH RISK (above the threshold).

5.3 Testing

Thirty-five automated tests check, among other things, that simulated readings stay inside the training ranges, that the simulator is deterministic and resets correctly, that an overloaded machine raises a HIGH RISK alert while a lightly loaded one does not, that replacing a worn tool lowers the risk, that batch and single predictions agree, that no leakage column is used as a model input, and that the shipped model beats its baseline.

6. Results and Analysis

6.1 Cross-validated model comparison

The table below reports 5-fold cross-validation on the training split. Engineered features help every model, and help tree ensembles the most: Gradient Boosting improves from a PR-AUC of 0.801 to 0.904 and its F1 from 0.704 to 0.896. Logistic Regression improves little because its precision stays very low when the class weights are balanced.

Table 5: Five-fold cross-validation on the training split

Model	PR-AUC (mean ± std)	Recall	Precision	F1
Gradient Boosting (+features)	0.904 ± 0.070	0.833	0.971	0.896
Random Forest (+features)	0.898 ± 0.042	0.794	0.943	0.860
Gradient Boosting (raw)	0.801 ± 0.052	0.592	0.875	0.704
Random Forest (raw)	0.726 ± 0.071	0.503	0.761	0.600
Logistic Regression (+features)	0.489 ± 0.063	0.838	0.167	0.278
Logistic Regression (raw)	0.451 ± 0.089	0.803	0.141	0.240
Dummy baseline	0.034 ± 0.000	0.000	0.000	0.000

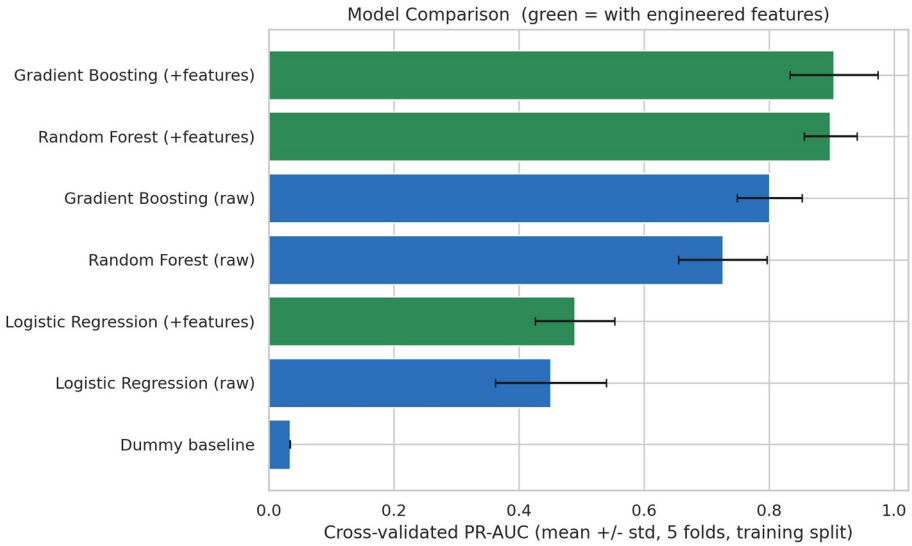


Figure 7: Cross-validated PR-AUC of all candidate models (green: with engineered features).

6.2 Final held-out test results

On the untouched test set (2,000 records, 68 failures) the final model at the balanced threshold detects 57 of 68 failures with only 2 false alarms. The baseline, an untuned Gradient Boosting model on raw features under the same protocol, detects 50 with 15 false alarms.

Table 6: Held-out test results

Model / mode	Thr.	Precision	Recall	F1	PR-AUC	TN/FP/FN/TP
Final model: balanced	0.32	96.6%	83.8%	89.8%	0.901	1930/2/11/57
Final model: high-recall	0.05	56.7%	86.8%	68.6%	0.901	1887/45/9/59
Baseline (raw, untuned)	0.35	76.9%	73.5%	75.2%	0.782	1917/15/18/50

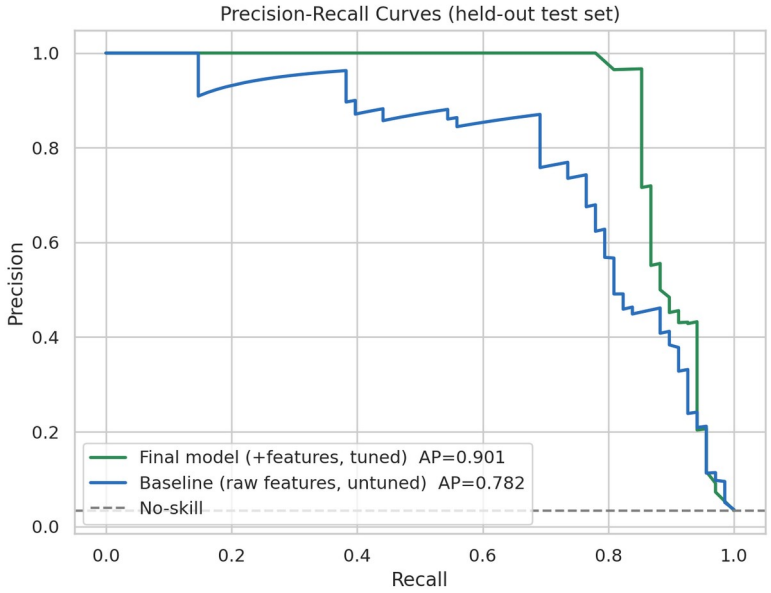


Figure 8: Precision–recall curves on the test set: final model versus baseline.

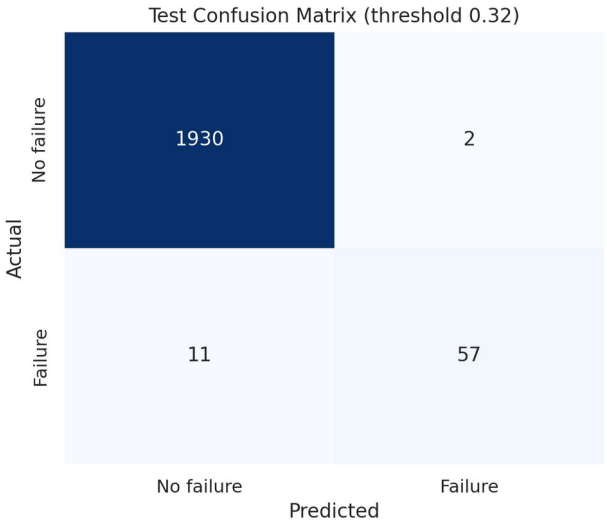


Figure 9: Confusion matrix of the final model at the balanced threshold.

6.3 Statistical uncertainty

Because the test set contains only 68 failures, 1,000 bootstrap resamples were used to quantify uncertainty. At the balanced threshold, precision is 0.966 (95% CI 0.911–1.000), recall 0.838 (0.741–0.921), F1 0.898 (0.835–0.947) and PR-AUC 0.901 (0.830–0.956). The results are therefore best read as “F1 of about 0.90”, not as exact values. The baseline's F1 of 0.752 lies below the lower bound of the final model's F1 interval, so the improvement is not a sampling accident.

6.4 Threshold trade-off

The threshold converts probabilities into alerts and is a business decision. The balanced mode gives the best precision–recall balance. The high-recall mode misses two fewer failures (9 versus 11) but raises 45 false alarms instead of 2, cutting precision to 56.7%. Under an assumed cost of 10 per missed failure and 1 per false alarm, the balanced mode has the lowest test cost (112 versus 135, and 195 for the baseline). A factory with very costly failures could still prefer the high-recall mode, which is why the dashboard offers both.

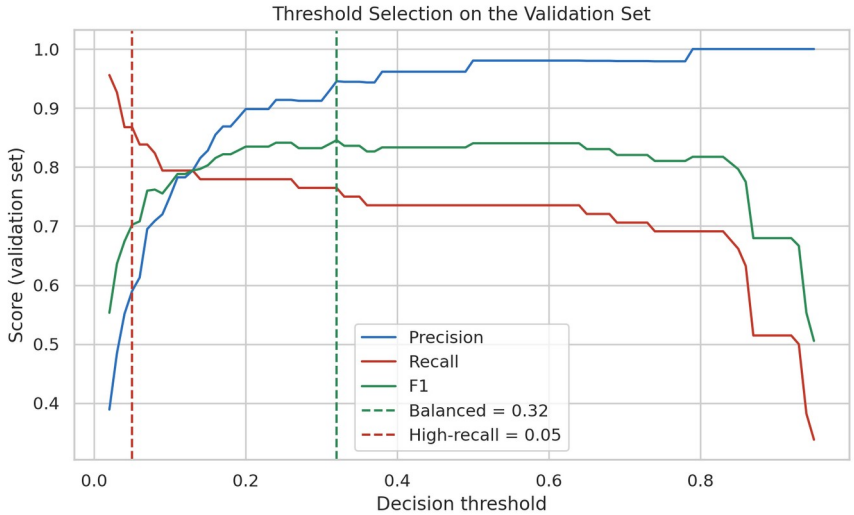


Figure 10: Precision, recall and F1 against the threshold on the validation set.

6.5 Calibration

The calibration curve compares predicted probabilities with observed failure frequencies. At the low end, where about 95% of machines sit, the model is well calibrated: 1,896 test machines scored up to 0.05 had a mean score of 0.5% and an observed failure rate of 0.5%. Scores between 0.05 and 0.2 were over-cautious (mean score 9%, observed 2.5%, 40 machines), and the few machines scored above 0.5 all failed (53 of 53), so high scores are, if anything, under-confident. Mid-range bins contain very few machines, so the scores should be treated as risk rankings rather than exact probabilities. This is why the dashboard calls its output a “predicted failure score”.

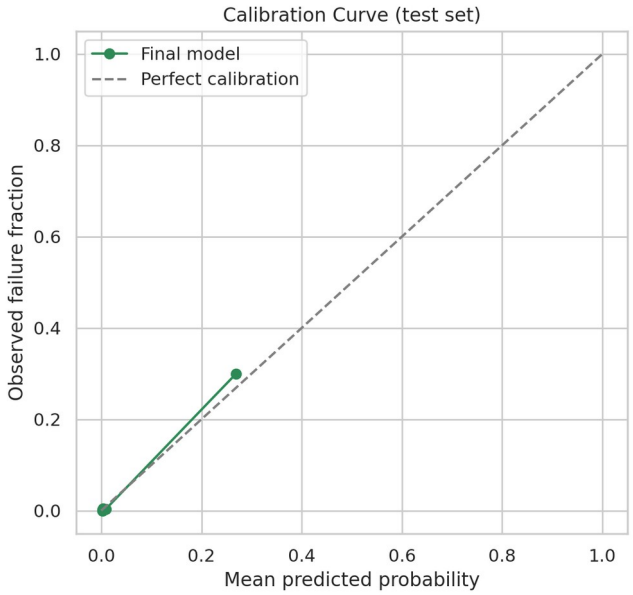


Figure 11: Calibration curve on the test set.

6.6 Feature importance

Permutation importance on the raw sensors (computed on the validation set) ranks torque first, followed by rotational speed and tool wear. Inside the tuned model, the three engineered features carry about 84% of the impurity-based importance (overstrain 30%, power 29%, temperature difference 24%), confirming that the model relies on the physical combinations rather than on individual readings.

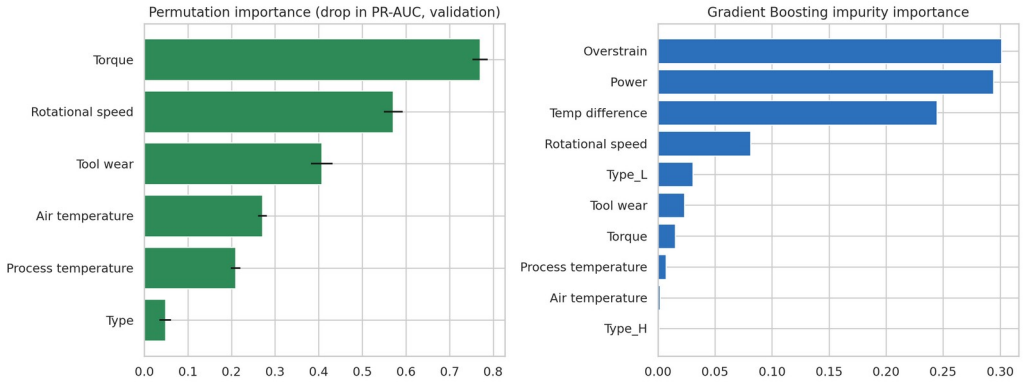


Figure 12: Left: permutation importance of the raw sensors. Right: impurity importance inside the model.

6.7 Error analysis by failure mode

Breaking the test-set recall down by documented failure mode explains the remaining 11 missed failures.

Table 7: Recall by failure mode (balanced threshold)

Failure mode	Failures in test set	Detected	Recall
PWF	13	13	100%
OSF	16	16	100%
HDF	29	28	97%
TWF	10	2	20%
RNF	4	0	0%

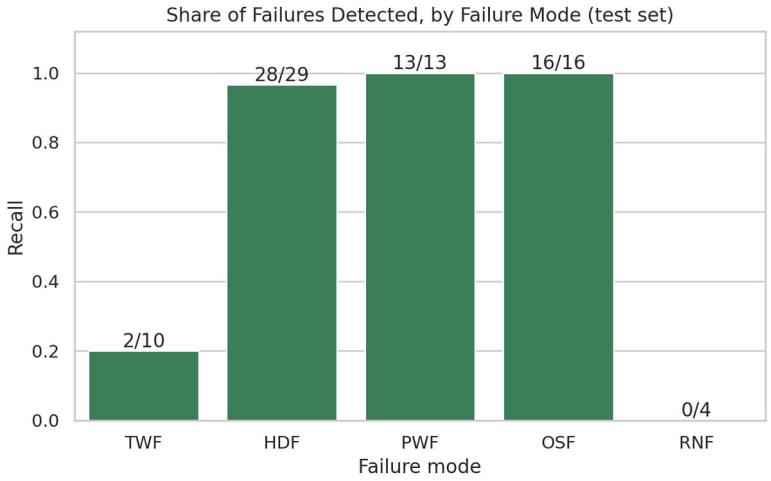


Figure 13: Share of failures detected, by failure mode.

Power (100%), overstrain (100%) and heat-dissipation failures (97%) are detected almost perfectly. The misses are concentrated in random failures (0 of 4) and tool-wear failures (2 of 10). These are produced in the data by chance or by a randomly drawn wear level, so no sensor-based model should be expected to predict them. The remaining headroom of the model is therefore small.

7. Discussion

7.1 What worked

- Domain-driven features produced the largest gain, larger than model choice or tuning. Encoding what the physics says about failures was more valuable than a more complex algorithm.
- A strict train/validation/test protocol and bootstrap intervals make the reported performance trustworthy and honest about its uncertainty.
- Reporting recall by failure mode turns a single score into an explanation of what the model can and cannot do.

7.2 Limitations

- The live sensor stream in the dashboard is simulated, and the dataset itself is synthetic. Real equipment would show noise, drift and failure patterns that are not represented here.
- With 68 test failures the estimates are uncertain (Section 6.3), and the final evaluation relies on a single split.
- The warnings shown to operators use the dataset's published failure rules; they explain alerts but are not model inputs.
- The cost ratio of 10:1 is an assumption; real maintenance costs would change the preferred threshold.

7.3 Future work

- Validate on real machine telemetry and monitor for data drift.
- Extend the task to remaining-useful-life estimation with survival or sequence models.
- Add per-prediction explanations (for example SHAP) and learn the threshold from real maintenance costs.

8. Conclusion

The project delivers a complete, reproducible predictive-maintenance system. Physics-based feature engineering and tuning raised the held-out F1 from 75.2% to 89.8% and PR-AUC from 0.782 to 0.901 with no data leakage. Failures that have a physical signature are detected almost perfectly, and the remaining errors are concentrated in failure modes that are random by design. The factory simulation and dashboard show the model working on live-style data, explain each alert, and let the user trade false alarms against missed failures. The main caveats are that the data are synthetic and the test set is small.

References

[1] Matzka, S. (2020). Explainable Artificial Intelligence for Predictive Maintenance Applications. Third International Conference on Artificial Intelligence for Industries (AI4I), IEEE, pp. 69–74.

[2] UCI Machine Learning Repository. AI4I 2020 Predictive Maintenance Dataset (id 601). https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset

[3] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[4] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. Annals of Statistics, 29(5), 1189–1232.

[5] Saito, T. and Rehmsmeier, M. (2015). The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. PLoS ONE, 10(3), e0118432.

[6] Streamlit documentation. https://docs.streamlit.io